PRACTICAL 10:

Practical10(1)

Problem Phase:

A parking lot has 10 slots numbered 0 to 9. Each vehicle is assigned a slot based on the last digit of its registration number. If the assigned slot is occupied, the solution checks the next slot sequentially and wraps around to slot 0 after slot 9. The goal is to display the final state of all slots after parking all vehicles.

Solution:

The solution uses a hash table with linear probing to assign vehicles to parking slots. First, it calculates the last digit of each registration number to determine the initial slot. If that slot is occupied, it checks the next slot until an empty slot is found. If it reaches the end, it wraps around to slot 0. When the parking lot is nearly full, many vehicles may need to check several slots, increasing the time required. If all 10 slots are occupied, the algorithm could loop indefinitely unless it checks whether all slots have been examined. Therefore, the solution should check at most 10 slots for each vehicle and report that the parking lot is full if no empty slot is found.

Practical10(2)

Problem Phase:

A library has 10 shelves numbered 0 to 9. Each book is assigned to a shelf by calculating book code % 10. Multiple books can be stored on the same shelf using a stack or linked list. The goal is to place all books on their correct shelves and display the final contents of all ten shelves.

Solution:

The solution uses a hash table with separate chaining to store books. The shelf number is calculated using book code % 10. If two or more books map to the same shelf, they are added to that shelf's list without replacing existing books. As more books accumulate on one shelf, searching becomes slower because more book codes must be checked. Normally, books are distributed across different shelves, but if all book codes have the same remainder when divided by 10, all books will end up on the same shelf.

Practical10(3)

Problem Phase:

A university stores student IDs in a hash table using a two-step hashing rule. The first hash function determines the initial slot. If that slot is occupied, a second hash function calculates the jump size to find another slot. The goal is to insert all student IDs into the table and display its final state.

Solution:

The solution uses double hashing to resolve collisions. The first hash function determines the initial index, and the second hash function determines the jump size whenever a collision occurs. The next slot is calculated using both hash functions and the probe number. If the jump size and table size have a common factor, some slots may never be visited, causing insertion to fail even when empty slots remain. Therefore, the jump size should be chosen so that it is relatively prime to the table size. Unlike Problem 10.1, which uses linear probing and checks consecutive slots, double hashing uses different jump sizes for different student IDs, helping to reduce clustering and distribute collisions more evenly.